# Synthesis Techniques and Intro to Faust

Intro to Audio Tech 1
Jocelyn Kavanagh
Monday, September 29

Georgia Tech

## Lesson Outline

- Sound Synthesis
- Block Diagrams
- Faust
- Up Next…

Georgia Tech

# Sound Synthesis

# Sound Synthesis Techniques

- Sample-based synthesis
- Wavetable synthesis
- Subtractive synthesis
- Additive synthesis
- Granular synthesis
- Amplitude Modulation
- Frequency Modulation
- Spectral Modeling
- Physical Modeling
- Non-standard synthesis

# Sound Synthesis Techniques

- Algorithms used to create sound
- 4 general categories:
  - Processed recording
    - time-domain manipulation of existing data
  - Spectral modeling
    - frequency-domain synthesis from analysis
  - Physical modeling
    - models of actual sources and sound propagation from acoustics
  - Abstract/loose modeling
    - any algorithm whose output can be put in audio range

# Sound Synthesis Techniques

- We can compare them based on
  - Sound quality
  - Flexibility
  - Generality
  - Computational Efficiency
- Most of the time we combine multiple techniques together
- Remember timbre involves envelope and spectra
  - Interesting sounds have unique spectra and change over time

For musical purposes, in the class ANY SOUND, almost all timbres are uninteresting, and many timbres feeble or ugly. ... It is VERY HARD to create new timbres we hear as interesting, powerful and beautiful.

- Max Mathews

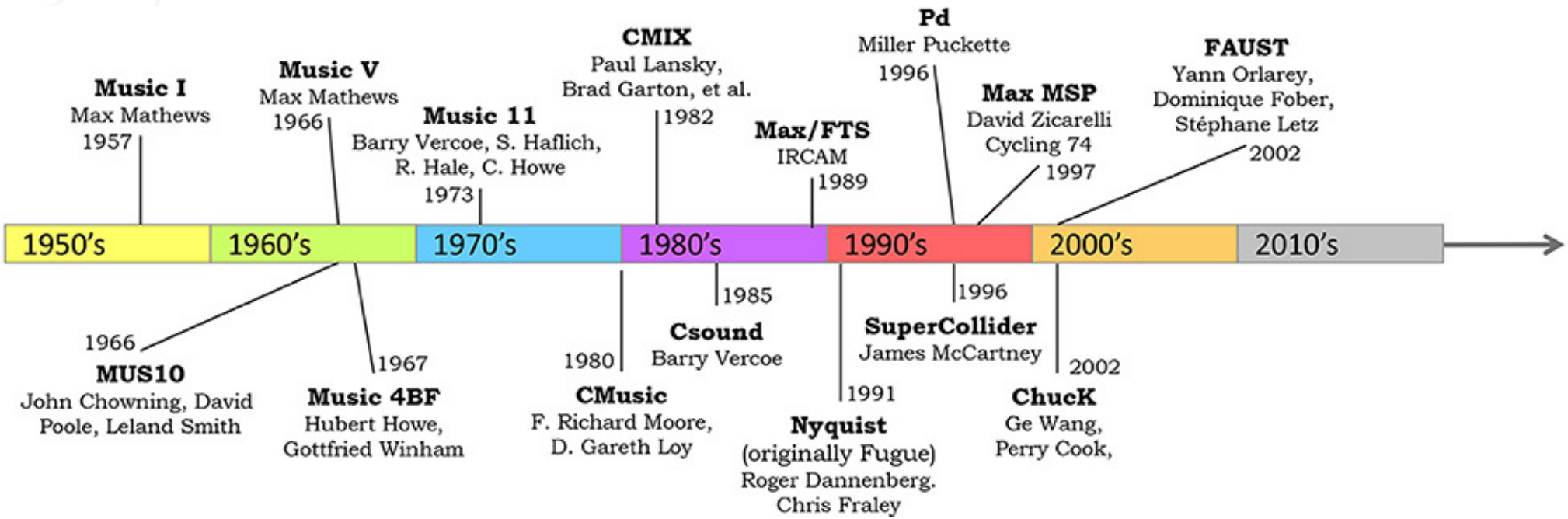
Georgia Tech

# Aside: Audio Programming Languages

# Music-N

- Max Mathews at Bell Labs developed MUSIC I in 1957
  - Earliest programming “language” for sound synthesis
  - first circuitry to convert digital data into analog sound
- Many iterations on MUSIC and these iterations were passed between studios and universities
- The ideas developed for the MUSIC-N languages persist in successor languages
  - Unit Generator (UGen)
    - Pre-defined building blocks for generating/controlling signal
  - Orchestra and Score
    - The connection of the ugens creates the orchestra which requires input of time-stamped signal changes called the score

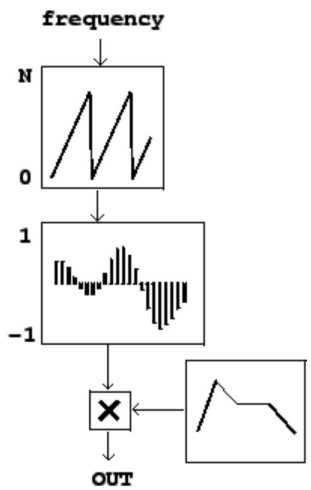
# Music-N Languages

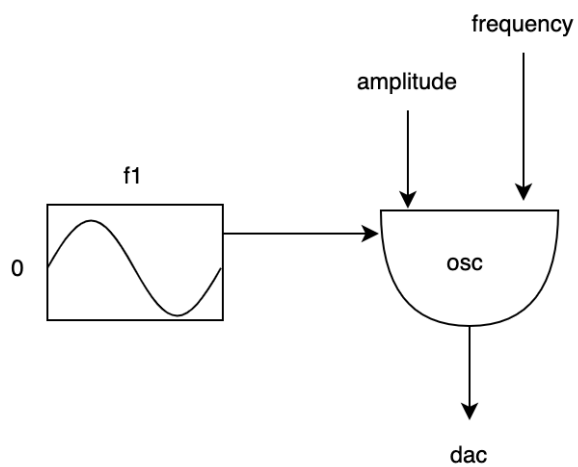


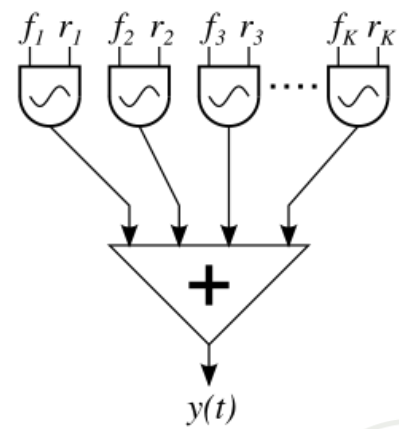
# Block Diagrams

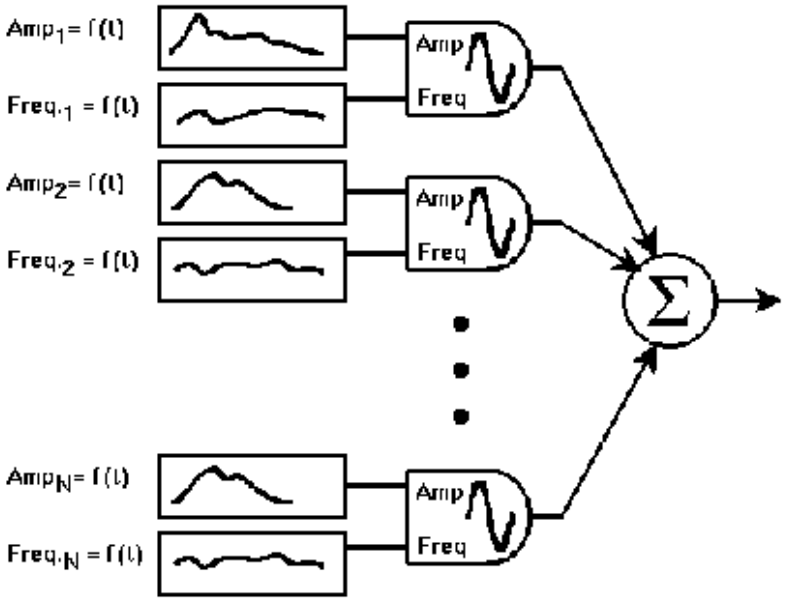
# Block Diagram / Schematics / Patch Diagram

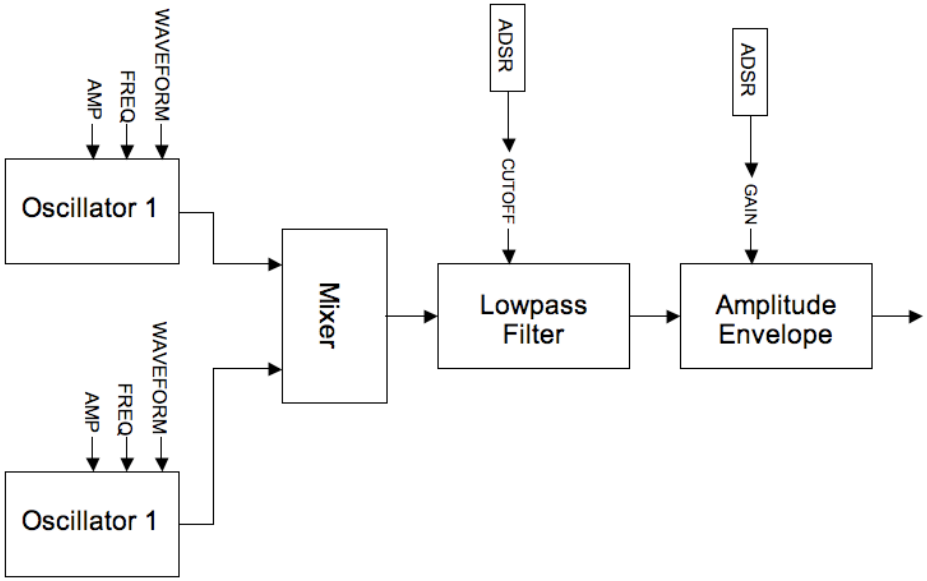
- When designing a software synthesis algorithm, we can draw a diagram of our patch to better understand signal/control flow
- There isn't really a "standard" way to draw these, but here are a few examples




Amp1 = f(t)
Freq.1 = f(t)
Amp2 = f(t)
Freq.2 = f(t)
AmpN = f(t)
Freq.N = f(t)
Amp
Freq
Amp
Freq
Amp
Freq
Σ


WAVEFORM
FREQ
AMP
Oscillator 1
WAVEFORM
FREQ
AMP
Oscillator 1
Mixer
ADSR
CUTOFF
Lowpass Filter
ADSR
GAIN
Amplitude Envelope

# Faust

# Why Faust?

- Functional programming language for sound synthesis and audio processing
- MUSIC-N language based around ugens
- Offers a text-based modality to supplement Max
- Can be an alternative/stepping-stone to C++
- Offers low-level options and higher-level abstractions
- Like Max, we can build code based on block diagrams

- Note: YOU WILL NOT BE TESTED ON FAUST. It is just used as a supplement in lecture

# Some Faust Syntax

- Navigate to https://faustide.grame.fr/
  - This is the online environment to work on Faust
- Import the Faust library
  - This allows us to use the built-in functions (kind of like Max objects)

```
import("stdfaust.lib");
```

- We end each program with the process declaration (kind of like main if you've used C)

```
process = (whatever we want our output)
```

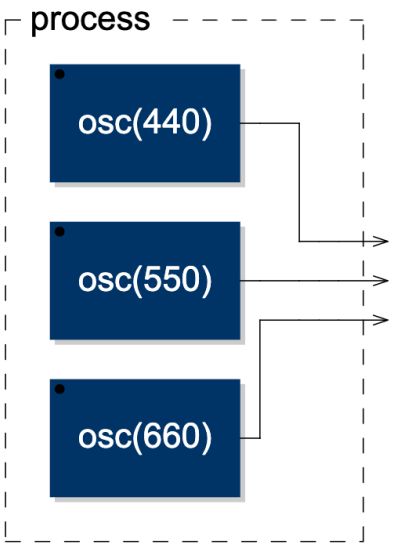
- We use diagram composition operations to control signals
  - Recursion ~
  - Parallel ,
  - Sequential :
  - Split <:
  - Merge :>

Georgia Tech

# Parallel and Sequential Composition

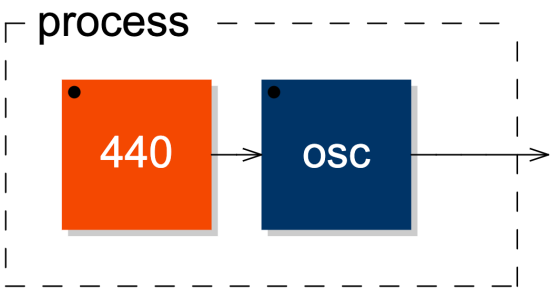
process = os.osc(440), os.osc(550), os.osc(660);

- Stacks signals on top of each other




process = 440 : os.osc;

- Output to input

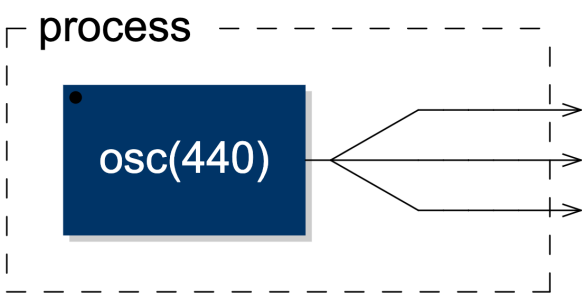



Georgia Tech

# Split and Merge Composition

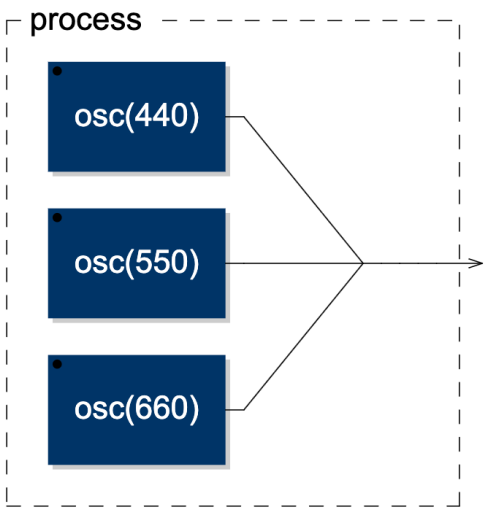
process = os.osc(440) <: _ , _ , _

- Distributes the output to a number of inputs




process = os.osc(440), os.osc(550), os.osc(660) :> _;

- Combines inputs to output




Georgia Tech

# Building an additive synthesizer in Faust

Georgia Tech

# Up Next…